PS1 Team Split: Smart Knowledge Base for Agents

CognitionNext '26 • two roles, one contract

Read both roles, then decide who takes which. The full architecture is in `reports/Novel RAG architecture for PS1.md` in the repo.

Budget and constraints

- **OpenCode credits: $10.** One full official evaluation over 100 questions costs about $1.5–2 (about 4M judge tokens). Generating the answers adds about $0.30 per 100 questions.
- Cache every judge call by its input hash, so nothing is paid for twice.
- Tune on a fixed, stratified 25-question subset (about $0.40 per run). Run the full 100 only 2–3 times: oracle, mid-build and final.
- Person 1 tunes against a **free offline proxy**: fuzzy text overlap with the gold chunks, with no LLM calls.
- Serving is **CPU only**: bge-small int8 embeddings and Tesseract OCR; no bge-m3 and no vision-language parsers. A cross-encoder used as a *feature* is allowed (confirmed by the PS author).
- Model: `deepseek-v4.1-flash` (paid, thinking disabled), as recommended by the PS author.

Person 1: Evidence Engine (retrieval, layers 1–4)

Goal: given `(collection, query)`, return the right set of paragraphs: correct documents and versions, and as many as the question needs, each with a calibrated relevance probability.

Owns

- **Ingestion:** PyMuPDF page text, with OCR for pages that have no text layer; stripping repeated headers and footers; paragraph units of about 63 words plus markdown tables; doc and page metadata on every unit.
- **Catalogue:** a document card for each document (entity, type, period, jurisdiction); MinHash version families; detection of consolidated texts; flags for units that differ between versions.
- **Indexes:** BM25 plus bge-small vectors.
- **Relevance model:** a features table; LightGBM with folds grouped by question; Platt calibration; the $\hat{G}$ estimate (expected number of gold chunks).
- **Set selector:** expected-utility cutoff k, a recall floor, and at least one unit per facet.

Milestones

Hour	Milestone
2	Units for the whole corpus, with a word-count histogram matching gold
5	A candidate scorer working end to end
8	Calibrated probabilities, $\hat{G}$ and set selection
10	Ablations, then freeze

Measured by: the free gold-overlap proxy (precision and recall against the gold chunks) on held-out folds. Official context precision and recall at the checkpoints.

Suits someone who likes: ML and information retrieval, feature engineering and data wrangling. Mostly offline work, with few LLM calls.

Person 2: Answer & Platform (generation, API, evaluation, layers 5–6)

Goal: turn a selected evidence set into a gold-style answer the judge scores highly, and own everything the organisers touch.

Owns

- **Evaluation harness:** the official notebook class imported unmodified, a batch runner, the judge cache, the budget tracker and the fixed 25-question subset.
- **Oracle run (E1):** feed the gold chunks to the writer and tune the prompt until the answer metrics stop improving. That measures the ceiling.
- **Query planner:** one cheap call that returns JSON with the entities, versions/dates, facets and answer type.
- **Writer and claim gate:** evidence grouped by facet; answer length set by $\hat{G}$; number and entity checks; abstention with the exact gold string (*"No information was retrieved for this question."*).
- **Platform:** FastAPI `/index` and `/query`; the required `trace` array (`step_name`, `description`, `input`, `output`); Langfuse cost and latency with manual usage details; deployment on their server; the stated ingestion time.
- **Write-up:** the 2–3 pages and the Langfuse export.

Milestones

Hour	Milestone
1	Harness working; E0 baseline scored
3	Oracle ceiling measured and the writer prompt tuned
6	API skeleton live with trace and Langfuse, using Person 1's stub
9	Full pipeline end to end
11	Deployed, final evaluation run, write-up done

Measured by: official answer precision and recall, first on the oracle and then end to end. Plus the API must be live and its responses must have the right shape.

Suits someone who likes: LLM prompting, backend and API work, deployment and writing.

The contract between the two

Agree this in the first 30 minutes, so both people can build against stubs.

```
Unit = {"id", "doc", "pages": [int], "type": "prose|table|formula|visual",
        "text", "section_path", "family_id", "version_role"}

engine.index(zip_path, collection) -> IndexStats
engine.select(collection, query, plan) -> {"units": [Unit + "p"], "G_hat": float,
                                           "facets": {facet: [unit_id]}, "trace": [step]}
```

The `plan` comes from Person 2's query planner, so Person 1 doesn't have to parse queries. Each side's `trace` steps are merged into the API response.

How to choose

Whoever is stronger in ML takes Person 1. Person 2 carries more of the scored-by-judges surface (API, deployment, write-up), so give it to whoever is better at shipping under time pressure.